SC — Stream Cine

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.7

Campo	Descrição
Projeto	SC — Stream Cine
Gerente de Projeto	Iasmin Cristina Krueger
Gerente/Responsável de GCS-DevOps	Iasmin Cristina Krueger
Fábrica de Software / Squad	Individual — Iasmin Cristina Krueger
Repositório principal	https://github.com/Iasminkrueger/stream_cine

Histórico de Revisões

Data	Versão	Descrição	Autor
06/09/2026	1.0	Registro inicial do projeto e adaptação do plano ao Stream Cine.	Iasmin Cristina Krueger
22/09/2026	1.1	Adaptação do plano ao projeto Stream Cine e às práticas efetivamente utilizadas.	Iasmin Cristina Krueger
23/09/2026	1.2	Adição do campo favorito em cada filme e ajuste da lógica de favoritar.	Iasmin Cristina Krueger
23/09/2026	1.3	Implementação da barra de pesquisa por nome do filme.	Iasmin Cristina Krueger
23/09/2026	1.4	Adição de pipeline de integração contínua (GitHub Actions) e revisão de dependências em Pull Requests.	Iasmin Cristina Krueger

Data	Versão	Descrição	Autor
14/10/2026	1.5	Ajustes de configuração do repositório (.gitignore e .gitattributes) e configuração do Dependabot para atualização automática de dependências.	Iasmin Cristina Krueger
16/10/2026	1.6	Atualização automática da dependência cupertino_icons via Pull Request do Dependabot; configuração de colaboração do repositório (CODEOWNERS, modelos de Issue e de Pull Request, README atualizado).	Iasmin Cristina Krueger
25/10/2026	1.7	Adição do par de revisão, dos environments staging e production, das regras de merge e da proteção da branch main (Aula 1); estrutura do repositório em formato de árvore; registro do .env.example como não aplicável; inclusão do plano na pasta docs/ e link no README.	Iasmin Cristina Krueger

Sumário

Sumário 3
1. Introdução 5
1.1 Finalidade 5
1.2 Escopo 5
1.3 Definições, Acrônimos e Abreviações 5
1.4 Referências 5
1.5 Visão Geral 6
2. Cultura DevOps e Papéis na Gerência de Configuração 6
3. Ferramentas, Ambientes e Infraestrutura 6
3.1 Ferramentas do Pipeline 6
3.2 Ambientes e Infraestrutura 7
3.3 Estratégia de Branching e Versionamento 8
4. Identificação da Configuração 8
4.1 Itens de Configuração 8
4.2 Convenção de Nomenclatura 9
4.3 Estrutura do Repositório 9
5. Integração Contínua (CI) 9
5.1 Etapas do Pipeline de CI 10
5.2 Política de Pull Request e Revisão de Código 10
5.3 Critérios de Qualidade (Quality Gate) 10
6. Entrega e Implantação Contínua (CD) 10
6.1 Etapas do Pipeline de CD 10
6.2 Estratégias de Implantação 10
6.3 Rollback e Recuperação 11
7. Infraestrutura como Código (IaC) 11
7.1 Versionamento de Infraestrutura 11
7.2 Ambientes Efêmeros 11
8. Controle de Configuração e Mudança 11
8.1 Processo de Solicitação de Mudança 11
8.2 Comitê de Controle de Mudança (CCM) 11
9. Baselines e Releases 11
9.1 Geração Automática de Baseline 11
9.2 Notas de Versão (Release Notes) 12
10. Estimativa do Status de Configuração 12
10.1 Monitoramento e Observabilidade 12
10.2 Métricas DORA 12
10.3 Relatórios e Auditorias 12
11. Marcos 12

12. Treinamento e Recursos 12
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros 13
14. Quadro-Resumo do Pipeline CI/CD 13

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC abrange a gestão dos itens de configuração de software ao longo do ciclo de desenvolvimento: código-fonte, dependências externas, arquivos de configuração do projeto, pipeline de integração contínua e o controle de versões via Git/GitHub. O plano cobre também o registro e o rastreamento de mudanças realizadas nesses itens, do commit inicial até o estado atual do repositório. Aplica-se a um desenvolvimento individual, sem ambientes de homologação ou produção.

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)

1.4 Referências

- Documento base: Processos de Desenvolvimento de Software e a Integração com GCS.
- Template do Plano de Gerenciamento de Configuração (RUP 7.0, IBM) — adaptado.

Referências específicas do projeto: código-fonte do Stream Cine, arquivo pubspec.yaml e repositório GitHub do projeto. O projeto utiliza as bibliotecas cached_network_image (exibição e cache de imagens dos filmes), cupertino_icons (ícones padrão do Flutter) e flutter_lints (análise estática de código), além da

GitHub Action dependency-review-action (revisão automática de dependências) e do Dependabot (atualização automática de dependências).

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Iasmin Cristina Krueger — define o escopo, organiza as atividades e acompanha a evolução do projeto.
Engenheiro DevOps / Release Engineer	Iasmin Cristina Krueger — realiza o versionamento Git/GitHub e a configuração do projeto. Configurou um pipeline simples de CI com GitHub Actions (análise estática de código e revisão de dependências) e o Dependabot.
Desenvolvedor(a)	Iasmin Cristina Krueger — desenvolve o aplicativo Flutter, implementa funcionalidades, integra a biblioteca e realiza testes manuais.
SRE / Operações	Não aplicável — não há ambiente de produção.
QA / Automação de Testes	Não há função separada. Os testes atuais são manuais realizados pela desenvolvedora.
Segurança (DevSecOps)	Não há função separada. Há um pipeline simples de verificação (GitHub Actions) e atualização automática de dependências (Dependabot), executados automaticamente.
Comitê de Controle de Mudanças (CCM)	Não utilizado, pois o projeto é individual e de pequeno porte.
Par de revisão (revisor)	juliasgr — colaboradora do repositório com permissão de escrita (Write); revisa e aprova os Pull Requests, já que o GitHub não permite aprovar o próprio PR.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas no projeto: Git, GitHub, GitHub Actions, Dependabot, Flutter, Dart, Visual Studio Code e pub.dev/cached_network_image.

Categoria	Ferramenta(s) sugerida(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento e armazenamento do código-fonte.
Orquestração de CI/CD	GitHub Actions	GitHub Actions, pipeline de CI configurado em .github/workflows/security.yml, com verificação de código (flutter analyze) e revisão de dependências em Pull Requests. CD não foi implementado.
Empacotamento	Flutter build	Geração de artefatos do aplicativo quando necessário.
Registro de artefatos	Não utilizado	O projeto não possui registro externo de artefatos.
Infraestrutura como Código	Não utilizada	Não há infraestrutura de servidores a provisionar.
Orquestração de containers	Não utilizada	O aplicativo não utiliza containers.
Gestão de segredos	Não utilizada	O projeto atual não possui credenciais de infraestrutura.
Monitoramento e Observabilidade	Não configurado	As verificações são realizadas manualmente durante a execução do aplicativo.
Gestão de dependências / SCA – Software Component Analysis	pubspec.yaml + pubspec.lock	Controle da dependência cached_network_image e demais pacotes Flutter.
Rastreamento de tarefas	Não utilizado	GitHub Issues — foram criados modelos de issue para funcionalidades e defeitos (.github/ISSUE_TEMPLATE).

3.2 Ambientes e Infraestrutura

O projeto utiliza um ambiente local de desenvolvimento para execução e testes do aplicativo Flutter. Não foram utilizados ambientes de homologação ou produção, nem Infraestrutura como Código (IaC), Docker ou Kubernetes.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento	Ambiente local para desenvolvimento, execução e testes do aplicativo Flutter.	Execução manual do aplicativo após alterações.
Homologação / Staging	Não utilizado no projeto atual.	Não aplicável.
Produção	Não utilizado no projeto atual.	Não aplicável.

Ambiente de desenvolvimento local: Windows, Flutter SDK e Android Emulator/Chrome para execução e testes. O aplicativo não utiliza banco de dados, servidor próprio, DNS, load balancer ou credenciais de infraestrutura.

Environments do GitHub (Settings → Environments): foram criados os environments staging (implantação restrita às branches protegidas) e production (revisora obrigatória: Iasmin Cristina Krueger, sem a opção

"Prevent self-review"; implantação restrita às branches protegidas). Eles foram preparados para o pipeline de entrega contínua previsto nas próximas etapas; por ora, não há implantação associada, e os ambientes de homologação e produção da tabela acima continuam sem uso.

3.3 Estratégia de Branching e Versionamento

No projeto atual foi utilizada uma estratégia simples de versionamento, com a branch principal main. Não foram criadas branches de feature, release ou hotfix, pois o desenvolvimento foi individual e de pequeno porte.

- main — branch principal utilizada para armazenar a versão atual do projeto. Executa o pipeline de CI (GitHub Actions) a cada push e Pull Request. Está protegida por regra de branch protection: Pull Request obrigatório com 1 aprovação, aprovações descartadas quando há novos commits, histórico linear e sem permissão de bypass. O arquivo CODEOWNERS define a responsável pelas revisões.
- Merge de Pull Requests — somente squash merge (merge commit e rebase desabilitados), com o título e a descrição do PR como mensagem do commit; as branches de origem são apagadas automaticamente após o merge. Cada PR resulta em um único commit na main.
- Par de revisão — o usuário juliasgr foi adicionado como colaboradora (permissão Write) e aprova os Pull Requests, pois o GitHub não permite aprovar o próprio PR.
- Branches de feature — não utilizadas neste projeto.
- Branches de release — não utilizadas neste projeto.
- Branches de hotfix — não utilizadas, pois o projeto não possui ambiente de produção.

O projeto possui versionamento do código por meio do Git. Foi criada a tag de release v1.4.0, seguindo Versionamento Semântico (SemVer). O modelo de Pull Request do repositório recomenda o uso de Conventional Commits nos títulos dos PRs.

4. Identificação da Configuração

4.1 Itens de Configuração

Além do código-fonte, em um contexto DevOps os seguintes artefatos também são itens de configuração formais, versionados no mesmo repositório (ou repositórios compartilhados):

- Código-fonte da aplicação e dos testes automatizados
- Definições de pipeline como código (ex.: .github/workflows, Jenkinsfile, .gitlab-ci.yml)
- Dockerfiles e manifests de containers
- Scripts de Infraestrutura como Código (Terraform, Ansible, Helm charts, manifests Kubernetes)
- Arquivos de configuração por ambiente (variáveis, feature flags), exceto chaves secretas, que devem ficar ambientes seguros.
- Scripts de migração de banco de dados (N/A)
- Documentação técnica (README, ADRs — Architecture Decision Records)

No projeto Stream Cine, já estão versionados: o pipeline de CI (.github/workflows/security.yml), a configuração do Dependabot (.github/dependabot.yml), o arquivo CODEOWNERS, os modelos de Issue (.github/ISSUE_TEMPLATE) e de Pull Request, e o README. Os demais itens da lista (Dockerfiles, scripts de IaC, arquivos de ambiente, migrações de banco) não se aplicam ao projeto atual.

Também estão versionados o .gitignore (que ignora arquivos .env, mantendo a exceção para .env.example), o .gitattributes (* text=auto eol=lf, para evitar problemas de quebra de linha entre sistemas) e o pubspec.lock (versões exatas das dependências). O arquivo .env.example não se aplica ao projeto atual, pois o aplicativo não utiliza variáveis de ambiente nem chaves secretas.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch principal	main	main
Branch de feature	Não utilizada	N/A
Commit	Mensagens descritivas no Git	Teste - Stream Cine
Tag de release	SemVer (vMAJOR.MINOR.PATCH)	v1.4.0
Imagem de container	Não utilizada	N/A

4.3 Estrutura do Repositório

A estrutura principal do repositório é apresentada a seguir:

```
stream_cine/
├── .github/
│   ├── ISSUE_TEMPLATE/
│   │   ├── defeito.yml                  modelo de Issue (defeito)
│   │   └── funcionalidade.yml           modelo de Issue (funcionalidade)
│   ├── workflows/
│   │   └── security.yml                 pipeline de CI (GitHub Actions)
│   ├── CODEOWNERS                       responsável pelas revisões
│   ├── dependabot.yml                   atualização automática de dependências
│   └── pull_request_template.md         modelo de Pull Request
├── android/                             plataforma gerada pelo Flutter
├── docs/
│   └── Gerenciamento_Configuracao_Stream_Cine.pdf   este plano (PGC)
├── ios/                                 plataforma gerada pelo Flutter
├── lib/
│   └── main.dart                        código principal do aplicativo
├── linux/                               plataforma gerada pelo Flutter
├── macos/                               plataforma gerada pelo Flutter
├── web/                                 plataforma gerada pelo Flutter
├── windows/                             plataforma gerada pelo Flutter
├── .gitattributes                       normalização de quebras de linha
├── .gitignore                           arquivos ignorados pelo Git
├── .metadata                            metadados do projeto Flutter
├── analysis_options.yaml                regras do flutter_lints
├── pubspec.lock                         versões travadas das dependências
├── pubspec.yaml                         dependências e configuração do projeto
└── README.md                            documentação do projeto
```

As pastas android, ios, linux, macos, web e windows são geradas automaticamente pelo Flutter. Não há diretórios de IaC. A suíte de testes padrão não faz parte da versão atual do projeto.

5. Integração Contínua (CI)

No estado atual do projeto, há um pipeline simples de Integração Contínua (CI), configurado em .github/workflows/security.yml, executado a cada push e Pull Request na branch main. Antes da proteção da branch main, a maior parte das alterações era enviada diretamente a ela; a partir da proteção, as alterações entram por Pull Request, revisado e aprovado pelo par de revisão. Pull Requests já foram utilizados em mudanças como a atualização automática de dependências pelo Dependabot e a configuração de colaboração do repositório.

5.1 Etapas do Pipeline de CI

1. Checkout do código e instalação das dependências (flutter pub get).
2. Build da aplicação (não implementado no pipeline atual).
3. Análise estática de código com flutter analyze.
4. Testes unitários (não implementado, pois o projeto não possui testes automatizados).
5. Testes de integração (não implementado).
6. Revisão automática de dependências (dependency-review-action) em Pull Requests, e atualização automática de dependências pelo Dependabot.
7. Empacotamento e publicação do artefato (não implementado).

5.2 Política de Pull Request e Revisão de Código

- Todo PR exige, no mínimo, 1 aprovação de outro desenvolvedor/colega antes do merge ("quatro olhos").
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente permitido com pipeline de CI verde (quality gate). No projeto atual, a exigência de status checks obrigatórios ainda não foi ativada na proteção da main; a regra vigente é de Pull Request com 1 aprovação. O arquivo CODEOWNERS define a responsável pela revisão. As regras acima são a política de referência para novos PRs.

5.3 Critérios de Qualidade (Quality Gate)

O pipeline de CI atual (security.yml) define dois critérios: o job de análise (flutter analyze) deve terminar sem erros; e a revisão de dependências, em Pull Requests, falha automaticamente se alguma dependência tiver vulnerabilidade de severidade alta (fail-on-severity: high). Não há critério de cobertura de testes, pois o projeto ainda não possui testes automatizados.

6. Entrega e Implantação Contínua (CD)

No estado atual do projeto, não há pipeline de Entrega/Implantação Contínua (CD) nem ambientes de staging ou produção. A execução do aplicativo é realizada manualmente no ambiente de desenvolvimento.

6.1 Etapas do Pipeline de CD

8. Deploy automático em ambiente de Homologação/Staging.
9. Execução de testes automatizados de aceitação/E2E em staging.
10. Gate de aprovação para produção (automático ou aprovação manual, conforme criticidade).
11. Deploy em produção com estratégia definida (ver 6.2).
12. Testes de fumaça (smoke tests) pós-deploy e verificação de saúde (health checks).
13. Geração automática de baseline/tag de release e notas de versão (changelog).

6.2 Estratégias de Implantação

Estratégia	Descrição
Rolling update	Não utilizada no projeto atual.
Blue-Green	Não utilizada no projeto atual.

Estratégia	Descrição
Canary Release	Não utilizada no projeto atual.
Feature Flags	Não utilizada no projeto atual.

6.3 Rollback e Recuperação

Como não há implantação em produção no projeto atual, não foi configurado um mecanismo automatizado de rollback. O histórico de commits do Git permite recuperar versões anteriores do código.

7. Infraestrutura como Código (IaC)

O projeto não possui infraestrutura de servidores ou serviços externos provisionados. Portanto, IaC não foi utilizada.

7.1 Versionamento de Infraestrutura

- Não aplicável ao projeto atual, pois não há infraestrutura como código.
- Não aplicável ao projeto atual, pois não há infraestrutura como código.

7.2 Ambientes Efêmeros

Não aplicável ao projeto atual, pois não existem ambientes de revisão provisionados automaticamente.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

No projeto atual, as mudanças são realizadas no ambiente local, testadas manualmente e registradas em commits Git. Como a branch main está protegida, elas chegam ao GitHub por Pull Request, executam o pipeline de CI, são aprovadas pelo par de revisão (juliasgr) e integradas por squash merge.

8.2 Comitê de Controle de Mudança (CCM)

Não foi utilizado Comitê de Controle de Mudanças (CCM), pois o projeto foi desenvolvido individualmente e não possui ambiente de produção ou mudanças de infraestrutura de alto impacto.

Não aplicável ao projeto atual.

9. Baselines e Releases

9.1 Geração Automática de Baseline

Não foram configuradas baselines ou tags de release automatizadas. O estado do projeto é rastreável pelo histórico de commits do Git.

9.2 Notas de Versão (Release Notes)

Não há geração automática de release notes, pois não foi configurado pipeline de release ou uso de Conventional Commits.

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

O projeto atual não possui ambientes de homologação ou produção monitorados. As verificações foram realizadas durante a execução e os testes manuais do aplicativo no ambiente de desenvolvimento.

10.2 Métricas DORA

Métrica	Descrição
Deployment Frequency	Não mensurada no projeto atual, pois não há pipeline CI/CD ou ambiente de produção.
Lead Time for Changes	Não mensurada no projeto atual, pois não há pipeline CI/CD ou ambiente de produção.
Change Failure Rate	Não mensurada no projeto atual, pois não há pipeline CI/CD ou ambiente de produção.
Mean Time to Restore (MTTR)	Não mensurada no projeto atual, pois não há pipeline CI/CD ou ambiente de produção.

10.3 Relatórios e Auditorias

No projeto atual, o histórico de commits do Git, o repositório GitHub e os logs de execução do pipeline de CI (aba Actions do GitHub) são os principais registros para rastrear alterações do código e verificações automáticas.

11. Marcos

Marcos do projeto: criação do projeto Flutter; implementação da interface do Stream Cine; inclusão da biblioteca cached_network_image versão 3.4.1; testes de execução do aplicativo; criação do repositório Git local; primeiro commit; publicação do projeto no GitHub; adição do campo de favorito aos filmes; implementação da barra de pesquisa por nome; configuração do pipeline de CI com GitHub Actions; criação da tag de release v1.4.0; configuração do Dependabot; configuração de colaboração do repositório (CODEOWNERS, modelos de Issue e de Pull Request, README atualizado). CD e IaC não foram implementados até o momento.

12. Treinamento e Recursos

O projeto foi desenvolvido individualmente por Iasmin Cristina Krueger. Foram utilizados Flutter, Dart, Visual Studio Code, Git, GitHub, GitHub Actions, Dependabot e a biblioteca cached_network_image. Para evoluir o pipeline de CI/CD, seriam necessários conhecimentos adicionais de testes automatizados e automação de build/deploy. IaC não é necessária para a versão atual do aplicativo.

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado lockfile).

- Análise automática de vulnerabilidades em dependências (SCA), feita pela GitHub Action dependency-review-action, executada em Pull Requests.
- Atualizações de dependências propostas automaticamente pelo Dependabot (.github/dependabot.yml), submetidas como Pull Request. Já foi aplicada uma atualização real: a dependência cupertino_icons foi atualizada de ^1.0.8 para ^2.0.0 por um PR automático do Dependabot.

A principal dependência externa utilizada é a biblioteca cached_network_image, adicionada ao pubspec.yaml na versão 3.4.1 e registrada no pubspec.lock. O projeto também utiliza a dependência cupertino_icons (versão ^2.0.0, atualizada pelo Dependabot) e, em desenvolvimento, o pacote flutter_lints (^6.0.0) para análise estática de código.

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
Controle de versão	Commit do código no Git e envio ao GitHub.	git commit / git push
CI	GitHub Actions: análise estática (flutter analyze), revisão de dependências e Dependabot.	push / pull_request
Merge	Já ocorreu em casos pontuais (ex.: Pull Request automático do Dependabot), seguindo o CODEOWNERS.	N/A
CD — Staging	Não configurado.	N/A
Gate de Release	Não configurado.	N/A